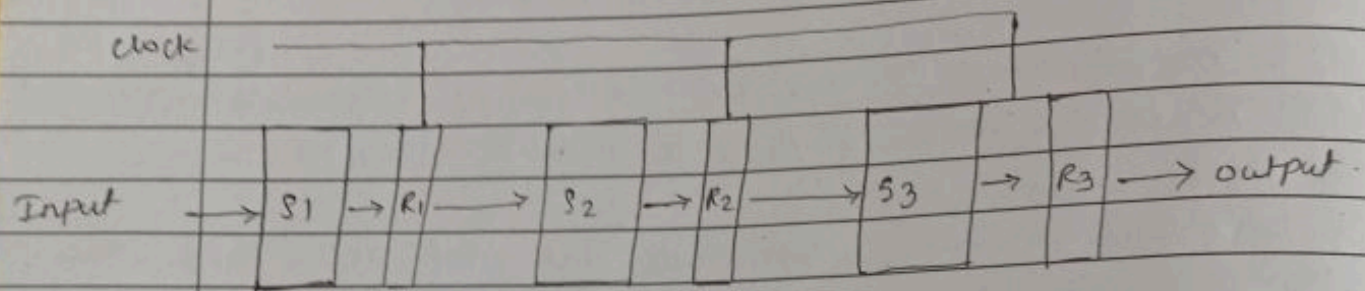


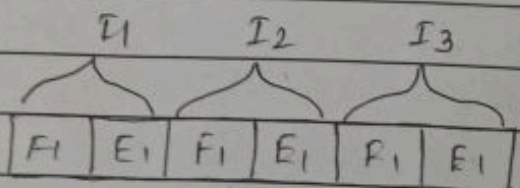
* Basic Concepts of Pipelining :-

- Pipelining creates and organizes a pipeline of instructions the processor can execute in parallel.
- The pipeline is divided into logical stages connected to each other to form a pipeline structure.
- Instructions enter from one end and exit from other.
- Pipelining is an ongoing, continuous process in which new instructions, or tasks are added to the pipeline and completed tasks are removed at a specific time after processing completes.
- In pipeline, each segment consists of an input register that holds data and a combinational circuit that performs operation.
- The output of the circuit is then applied to the input register of the next segment of the pipeline.
- Here are the steps in the process
 - ① Fetch instructions from memory
 - ② Read the input register, and ~~decode~~ decode the instruction.
 - ③ execute the instruction
 - ④ Access an operand in data memory
 - ⑤ Write the result of operation into the input register of the next segment.

- Pipeline is divided into stages and these stages are connected with one another to form a pipe like structure. Instructions enters from one end and exit from other end.

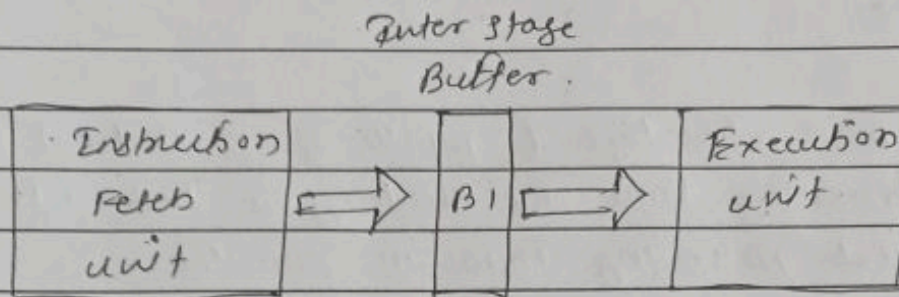


- In pipeline system, each segment consist of an input register followed by a combinational circuit.
- The register is used to hold data and combinational circuit performs operations on it.
- Assume processor executes a program by fetching and executing instructions one after the other.
- let F and E refers to fetch and executes steps for instruction I.
- Execution of program consists of a sequence of fetch and execute steps as follows.



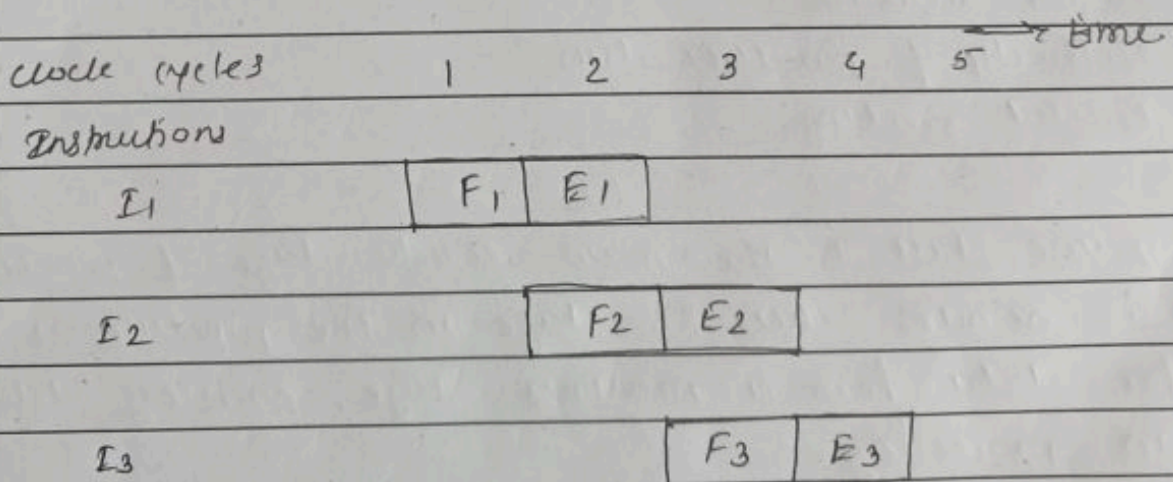
* Hardware organization of pipelining :-

considers computer that has two separate hardware units, one for fetching instruction and another for executing them.



* pipelined executions of Instructions :-

The computer is controlled by a clock whose period is such that the fetch and execute step of any instruction can be completed in one clock cycle.



* Advantages :-

- ① Faster Processing
- ② Improved Resource Utilization
- ③ Flexibility
- ④ Lower Latency
- ⑤ Scalability

- one of the main advantage of pipelining is that it can speedup processing time. By breaking a task into smaller subtasks and processing them in parallel.

- pipelining can improve the overall throughput of a system.

* Disadvantage:

- Pipeline stalls
- Increased complexity
- Higher overhead
- Difficulty of parallelization
- limited scaling.

- pipeline stall is the main disadvantage of pipelining, this occurs when a stage in the pipeline is waiting for data from a previous stage, causing delay in processing.

* Arithmetic an Instruction Pipeline :-

① Arithmetic Pipeline :-

Arithmetic pipeline represents the parts of an arithmetic operation that can be broken down and overlapped as they are performed.

- It can be used for arithmetic operations such as floating-point operations, multiplication of fixed-point numbers etc.
- Registers are used to store any 'intermediate results' that are then passed on to the next stage of further processing.

ex. addition of two floating point numbers.

$$x = A \times 2^a$$

$$y = B \times 2^b$$

Here,

A, B are the mantissa and a, b are the exponents.

1. check for zeros
2. Align the mantissa
3. Perform add or subtract mantissa
4. Normalize the result.

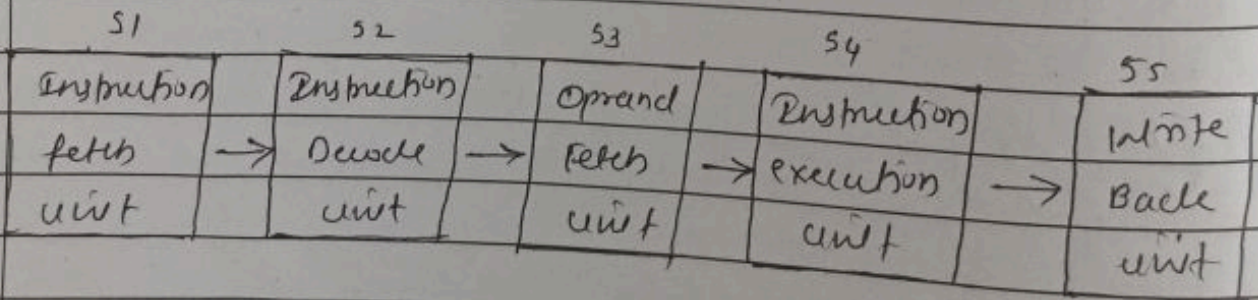
The registers are used to store intermediate results in above example/operations.

* Instruction pipeline :-

- In this stream of instructions can be executed by overlapping fetch, decode and execute phases of an instruction cycle.
- This type of technique is used to increase the throughput of a computer system.
- An instruction pipeline reads instruction from the memory while previous instructions are being executed in other segment of the pipeline.
- The pipelining will be more efficient if the instruction cycle is divided into segments of equal duration.
- In most general case computer needs to process each instruction in following sequence of steps.

- ① Fetch instruction from memory (FI)
- ② Decode the instruction (DA)
- ③ calculate effective address
- ④ Fetch the operands from memory (Fo)
- ⑤ Execute the instruction
- ⑥ Store the result in proper place.

७.



S ₁	1	2	3	4	5	6	7	8	9
S ₂		1	2	3	4	5	6	7	8
S ₃			1	2	3	4	5	6	7
S ₄				1	2	3	4	5	6
S ₅					1	2	3	4	5
	1	2	3	4	5	6	7	8	9 time →

* Throughput and speedup :-

- Latency is the time from start to finish for a given task.

• Throughput :-

- throughput is the number of tasks completed in a given time period.

- throughput is the outputs produced per clock cycle and that throughput will be equal to 1, in case of ideal situation that means, when the pipeline is producing one output per clock cycle.

$$\text{throughput} = \frac{\text{number of instruction executed}}{\text{execution time}}$$

* Speedup :-

Speedup is a number that measure the relative performance of two system processing the same problem.

- The speedup of a pipeline measures how much more quickly a workload is completed by the pipeline processor, than by a non-pipeline processor.

$$\text{Speedup (S)} = \frac{\text{Performance of non pipelined processor}}{\text{Performance of pipelined processor.}}$$

* Pipeline Hazards :-

- The problems that occur in the pipeline are called as hazards. Hazards that arise in the pipeline prevent the next instruction from executing during its designated clock cycle.

There are three types of hazards

1. Structural Hazards
2. Data Hazards
3. Control hazards.

- Because if these three hazards instructions get delayed.
i.e. we didn't get $CPI = 1$

- Structural hazards occurs when multiple instructions need the same resource. resources may be processor, memory, registers, RAM, cache etc.

eg. I_1 in write Back needs access of memory at the same time I_2 in memory needs to access memory.

I_1	IF	ID	EX	Mem	WB	
I_2		IF	ID	EX	Mem	WB.

- Data hazards occur when an instruction depends on the result of previous instruction and that result of instruction has not yet been computed.

There are three situations in which data hazards can occur. i.e. read after write (RAW), write after read (WAR) and write after write (WAW).

- All instructions who change the program counter leads to control hazards. the control hazards occur due to branches

Control hazards occurs whenever the pipeline makes incorrect branch prediction, decision, resulting in instruction entering the pipeline that must be discarded.

eg. JMP or Branch, Bne R_1 , 2000 ($R_1 \neq 0$, Branch)

* Logic Design Conversion :-

- Performance of computer is determined by three key factors:

- ① Instruction count
- ② clock cycle time
- ③ clock cycle per instruction (CPI)

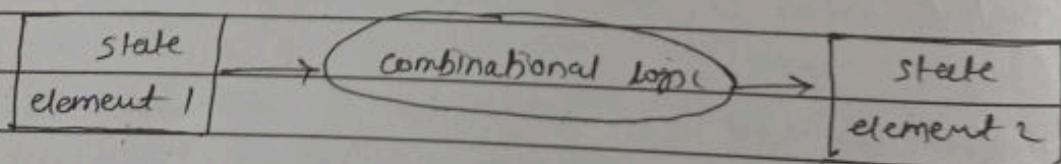
- To discuss design of computer, we must decide how the logic implementing the computer will operate and how the computer is clocked.

- below terms are used :

1. Asserted (assert) : The signal is logically high or true.
2. Deasserted (deassert) : The signal is logically low or false.

* clocking Methodology :-

- A clocking methodology defines when signals can be read or write.



clock
cycle.

- Combinational logic, state element and clock are closely related. The time necessary for the signals to reach state element 2 defines the length of the clock cycle.

- ~~data~~ control signal :-

A signal used for directing the operation of a functional unit.

- Data signal :-

A signal contains information that is operated by a functional unit.

- Bus :-

Bus is signals wider than 1 bit, with thicker line. Several buses combines to form a wider bus. for ex. 32-bit bus is obtained by combining two 16-bit buses.

Functional elements:-

Two types of functional elements in the hardware.

① Combinational elements:-

elements that operates on data.

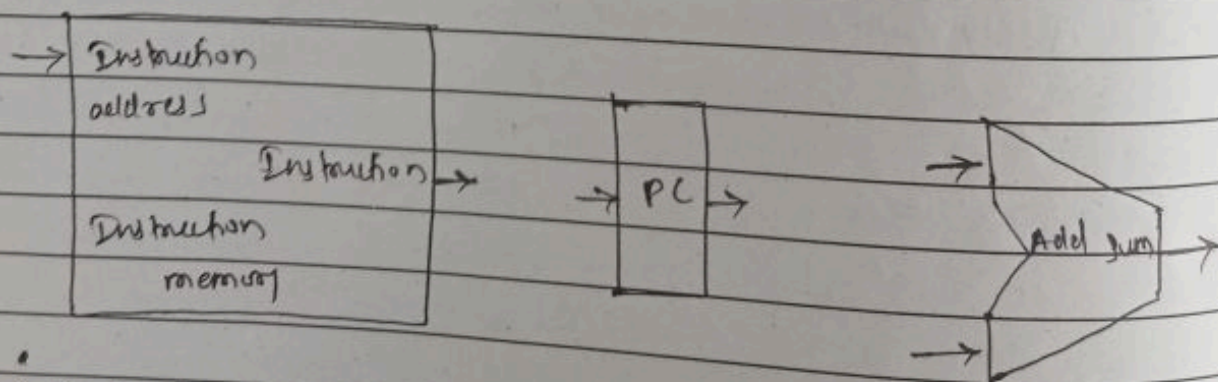
② Sequential / state elements:-

elements that contains data.

* Building a Data Path :-

- The CPU can be divided into Data section and a control section. Data section is also called as data path contains Registers and the ALU and can do certain operations on data item.
- Control section issues control signals to data path.
- A data path is a set of elements that process data and addresses in the CPU.
- To implement data path we have different approaches
 1. One-Bus organization
 2. Two-Bus organization
 3. Three-Bus organization.

For Building datapath we have Instruction memory, program counter, Adder.



a) Instruction memory

b. Program counter.

c. Adder

- while building datapath we have

1. Fetching Instruction
2. R-Format Instruction (Arithmetic-Logical Instruction) (ADD, SUB, AND, OR and SLT)
3. Load word and store word instruction
4. Branch Instruction (beq)

- one bus organization is simple and cheap in implementation but disadvantage will be, every one will try to access a single bus. i.e. general purpose registers, PC, IR, MAR, MDR, will try to access same bus.

- In two bus organizations we have two buses which can be used by everyone. Also we can say that two bus organization has two buses i.e. In-Bus and outp Bus.

- In three-Bus organization we have one In-bus. and two out Buses which will work faster as compare to one bus and two bus organization.

A Data Hazards : Forwarding versus Stalling :-

To understand what is stall consider below example.

	F	D	E	WB	clock cycle.
I ₁	1	2	1	1	=5
I ₂	1	2	2	1	=6
I ₃	2	1	3	2	=8
I ₄	1	3	2	1	=7
I ₅	1	2	1	2	=6
					32

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
I ₁	F	D	D	E	W														
I ₂		F	X	D	D	E	E	W											
I ₃			F	F	X	D	X	E	E	E	W	W							
I ₄					F	X	D	D	D	X	E	E	W						
I ₅						F	X	X	X	D	D	X	E	W	W				

- Here we achieve $32/15 = 2.1$ times faster performance. but still we get lot of stalls while executing instruction.
- Best way to increase the performance of system and to remove stalls in execution we can increase the time of clock cycle.
- Because of which we did not get a stall in the execution of instruction.

① Read after Write Hazard : (RAW)

			Domain	Range
I_1	$R_3 \leftarrow R_1 + R_2$	I_1	R_1, R_2	R_3
I_2	$R_5 \leftarrow R_3 + R_4$	I_2	R_3, R_4	R_5

$$I_1(\text{Range}) \cap I_2(\text{Domain}) \neq \emptyset \quad (\text{RAW hazard})$$

$R_3 \qquad R_3$

② Write after Read Hazard (WAR) :-

I_1 :	$R_1 \leftarrow R_2 * R_3$	Domain(I_1) :	R_2, R_3
I_2 :	$R_2 \leftarrow R_4 + R_5$	Domain(I_2) :	R_4, R_5
		Range(I_1) :	R_1
		Range(I_2) :	R_2

$$\text{Domain}(I_1) \cap \text{Range}(I_2) \neq \emptyset \quad \text{WAR hazard}$$

③ Write after Write Hazard (WAW) :-

I_1 :	$R_3 \leftarrow R_1 * R_2$
I_2 :	$R_3 \leftarrow R_4 + R_5$

$$\text{Domain}(I_1) = R_1, R_2$$

$$\text{Domain}(I_2) = R_4, R_5$$

$$\text{Range}(I_1) = R_3$$

$$\text{Range}(I_2) = R_3$$

$$\text{Range}(I_1) \cap \text{Range}(I_2) \neq \emptyset$$

* Forwarding in Data Hazards:-

operand forwarding is used to reduce the stall while executing instructions. Because of such type of hazards we are unable to maintain $CPI = 1$.

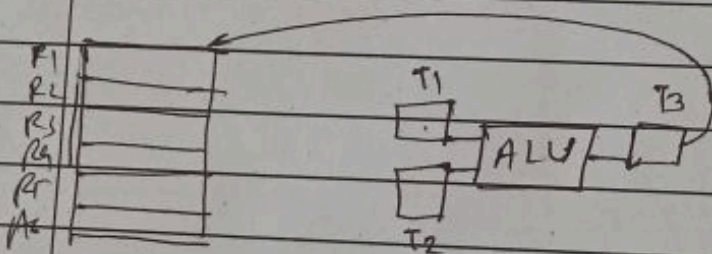
- Operand forwarding is used to reduce Read after Write (RAW) hazards in pipeline.

- To reduce WAR and WAW hazards we use register renaming.

Consider below instructions:

I1: ADD $R_3 \leftarrow R_1 + R_2$

I2: SUB $R_5 \leftarrow R_4 - R_3$



- A data hazard can lead to a pipeline stall when the current operation has to wait for the result of an earlier operation which has not yet finished.

Fetch	Decode	Read operand	Execute	Write
ADD	ADD	ADD	ADD	Result
Fetch	Decode	Stall	Stall	Read operand
SUB	SUB			Execute
				Result

- Forwarding is the concept of making data available to the input of the ALU for subsequent instructions, even though the generating instruction has not reached to WB in order to write the memory or register.

- This is also called as short circuiting or by passing.

and forwarding

Fetch	Decode	Read operand	Execute	Write	
ADD	ADD	ADD	ADD	Result.	
	Fetch	Decode	Read operands	execute	Write Result
	SUB	sub	SUB: use result from prev. op ⁿ .	SUB	

* Control Hazards :-

Consider a case where, Instruction I_1 is branch instruction and after executing the entire instruction we understand that there is a jump and next instruction to be executed is I_{10} . This time we have already partially executed I_2, I_3 , which is the problem.

- Control hazards occurs because of branch instruction.

The following types of instructions can introduce control hazards.

1. Unconditional branches
2. Conditional branches
3. Indirect branches
4. Procedure calls
5. Procedure returns.

* Solutions for control Hazards :-

1. Pipeline Stall cycle :-

freeze the pipeline until the branch outcomes and targets are known, then proceed with Fetch operation.

2. Branch Prediction :-

the outcome and target of conditional branches are predicted using some heuristic. But the results are not written back to register file or memory until the branch is executed and predictions are verified.

3. Reordering Instructions :-

Reorder the instruction to move the branch instruction later in the sequence, allowing safe and beneficial instructions that are unaffected by the result of a branch to be brought in earlier in the sequence, delaying fetch of the branch instruction.

- If such instructions are not available, Nop is used. the compiler is used to implement this delayed branch.

4. Dynamic branch Prediction :-

with the help of branch Table Buffer, a historical record is kept (BTB).

The BTB is a type of cache that contains a series of entries containing the branch instructions PC address and the effective branch address.

- when conditional branch instruction is encountered, the BTB is queried for the matching branch instruction address.
- If the target branch address is hit, the next instruction is fetched from the associated target branch address.

* Exceptions :-

Exceptions and interrupts are unexpected events which will disturb / disrupt the normal flow of execution of instruction.

- Exception is an unexpected event from within the processor.

- Interrupt is an unexpected event from outside the processor.

- whenever an exception or interrupt occurs, the hardware starts executing the code that performs an action in response to the exception.

- This action may involve killing a process, outputting an error message, communicating with an external device, or horribly crashing the entire computer system by initiating a "Blue Screen of Death" and halting the CPU.

- whenever an exception or interrupt occurs, execution transition from user mode to kernel mode where the exception or interrupt is handled.

While entering the kernel, the context of currently executing process must first be saved to memory.

The kernel is now ready to handle interrupt / Exception.

1. Determine the cause of the exception / Interrupt
2. Handle the exception / Interrupt.

- when the exception / Interrupt have been handled the kernel performs the following steps:

1. Select the process to restore and resume.
2. Restore the context of the selected process.
3. Resume execution of the selected process.

- Consider following types of event that cause exception / Interrupt.

1. I/O device request	External	Interrupt.
2. Invoke the OS from program	Internal	Exception
3. Arithmetic overflow	Internal	Exception
4. Using an undefined instruction	Internal	Exception.
5. Hardware Malfunctions	Either	Exception / Interrupt

* Parallelism via Instructions :-

ILP (Instruction-level parallelism) is the parallel or simultaneous execution of sequence of instructions in a computer program.

- ILP refers to the average number of instructions run per step of this parallel execution.

- There are two approaches to instruction level parallelism:

1. hardware (dynamic Parallelism)
2. Software (static Parallelism)

1. In hardware or dynamic parallelism the processor decides at run time which instructions to execute in parallel.

2. In software or static parallelism the compiler decides which instructions to execute in parallel.

eg. Pentium processor works on dynamic sequence of parallel execution and Itanium processor works on the static level parallelism.

consider the following example program / Instruction

1. $e = a + b$
2. $f = c + d$
3. $m = e * f$

- Here, operation 3 depends on result of operations 1 and 2 so it cannot be calculated until both of them are completed. However, operations 1 and 2 do not depend on any other operation, so they can be calculated simultaneously.

- If we assume that each operation can be completed in one unit of time then these three instructions can be completed in total of two units of time, giving an ILP of 3/2.

- ILP increases the depth of the pipeline to overlap more instructions.

- Launching multiple instructions per stage allows the instruction execution rate (CPI) to be less than 1.